Ders 7 Genişletilmiş Notu: Template Method, Visitor ve Mediator Kalıpları

Ders 7 Genişletilmiş Notu: Template Method, Visitor ve Mediator Kalıpları

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Template Method Kalıbı: Amaç ve Sorun
- 2. Template Method GoF Örneği: Belge Açan Çerçeve Program
- 3. Örnek Gerçekleme: Farklı Veritabanlarından Kayıt Çekme
- 4. Template Method ile Strategy Karşılaştırması
- 5. Visitor Kalıbı: Amaç, Sorun ve Eleştiri
- 6. Visitor GoF Örneği: Derleyici Ağacı
- 7. Visitor Kalıp Yapısı ve Zayıf Yönleri
- 8. Örnek Gerçekleme: Proje Yönetim Yazılımı, Çözüm 1
- 9. Örnek Gerçekleme: Proje Yönetim Yazılımı, Çözüm 2 (Visitor)
- 10. Mediator Kalıbı
- 11. Kalıp Şemalarında UML Gösterimi
- Örnek Sorular
- Dersin Toparlaması

1. Template Method Kalıbı: Amaç ve Sorun

Ders, davranışsal (behavioral) kalıpların son üçüyle sürer: Template Method, Visitor ve Mediator. Hoca, bu üç kalıbı bu derste bitirmeyi hedeflediğini söyleyerek Template Method ile başladı.

Template Method, kalıtımın getirdiği çok biçimliliğin ve metot override etme işleminin biraz "çekidüzen altına alınması", disipline edilmesiyle ilgili bir kalıptır. Slayttaki tanıma göre amaç, bir algoritmanın bazı adımlarını belirleyip bazılarının ayrıntılarının belirlenmesini ertelemektir (ekrandan, 00:05:20). Hocanın anlatımıyla: algoritmanın sabit kalması gereken ana adımları bir üst sınıfta belirlenir, değişebilecek ara adımların ayrıntıları ise alt sınıfların doldurmasına bırakılır.

Slayttaki örnek ve sorun şöyledir:

- Örnek: farklı belge türleriyle çalışabilecek uygulamalar geliştirmeyi sağlayan bir çerçeve program (framework) hazırlanıyor. Kullanıcı, çerçevenin sunduğu Belge ve Uygulama sınıflarından kalıtım ile yeni sınıflar türeterek kendi yazılımını hazırlıyor.
- Sorun: bir belge açar veya oluştururken her türlü ince ayrıntıyı kullanıcıya atmak istemiyoruz; bazı temel adımların uygun sıra ile yürütülmesini çerçeve programa bırakmak istiyoruz.

Hoca, farklı türden belgeler ve bunlarla çalışan farklı türden uygulamalar olsa da hepsinde ortak yapılabilecek işler olduğunu, bu işlerin alt sınıflarda tekrar tekrar yazılmasını önlemek için davranışı ortak bir üst sınıfta toplamanın yararlı olacağını söyledi.

2. Template Method GoF Örneği: Belge Açan Çerçeve Program

2.1 Sınıf şeması

Hoca slayttaki şemanın ortasından yatay bir çizgi çizdi: çizginin üstü framework, altı bizim yazacağımız yeni uygulamadır (ekrandan, 00:10:00). Şema metne şöyle dökülebilir:

```
Framework (çizginin üstü)
  Document (soyut)                 Application (soyut)
    Save()                           AddDocument()
    Open()                           OpenDocument()
    Close()                          DoCreateDocument()      (soyut)
    DoRead()        (soyut)          CanOpenDocument()       (soyut)
                                     AboutToOpenDocument()   (soyut)
  Application --docs--> Document  (parça-bütün)

Uygulama (çizginin altı)
  MyDocument : Document            MyApplication : Application
    DoRead()                         DoCreateDocument()   -> "return new MyDocument"
                                     CanOpenDocument()
                                     AboutToOpenDocument()
  MyApplication - - -> MyDocument  (oluşturma bağımlılığı)
```

Uygulamanın açacağı belgeler olduğundan Application ile Document arasında parça-bütün ilişkisi vardır. Her belge kaydedilir, açılır, kapatılır; bunlar standart işlerdir. Kaydetme, baytların diske yazılmasıdır. Açma işleminde bir dosya tutamacı (file handle) alınır; ama okunan baytların nasıl yorumlanacağı belge türüne göre değişir. Bu yüzden `DoRead()` soyut metottur (slaytta italik ve altı çizili), `Open()` ise somut bir metottur ve `DoRead()` onun içinden çalıştırılır. Hoca, Document ve Application sınıflarının da soyut olduğuna, adlarının italik yazılmasından anlaşıldığına dikkat çekti.

Uygulama tarafında yeni açılmış bir belgeyi listeye eklemek ya da bir belgeyi açmak belge türünden bağımsız işlerdir; bunlar parça-bütün ilişkisini yöneten işlerdir. Buna karşılık gerçekten yeni bir belge oluşturmak ya da bir belgenin açılıp açılamayacağını denetlemek belge türüne göre değişir. Hoca, buradaki denetimin dosya erişim hakkı değil, belgenin biçiminin uygun olup olmadığı, verinin tam mı bozuk mu olduğu gibi konular olduğunu vurguladı.

2.2 Template metot: OpenDocument

Slayttaki kod (ekrandan, 00:10:00):

```
void Application::OpenDocument (const char* name) {
    if (!CanOpenDocument(name)) {
        // cannot handle this document
        return;
    }
    Document* doc = DoCreateDocument();
    if (doc) {
        _docs->AddDocument(doc);
        AboutToOpenDocument(doc);
        doc->Open();
        doc->DoRead();
    }
}
```

Akış şudur: belgeyi açabilir miyim? Açamıyorsam bir istisna at ya da bildir. Açabiliyorsam belgeyi oluştur; işaretçi boş değilse belgeyi listeye ekle, açmadan önce ne yapılması gerekiyorsa yap, sonra aç ve oku. Üst sınıftaki somut metot, alt sınıfta kodlanacak soyut metotları çağırarak bu noktalarda alt sınıfın devreye girmesini sağlar.

2.3 Kalıtımdan farkı ve final

Soru: Bunun sıradan kalıtımdan farkı nedir? **Cevap:** Hocanın cevabının iki parçası var. Birincisi, burada tek bir sınıftan kalıtım değil, birbiriyle ilişkili iki ayrı sınıftaki iki ayrı kalıtım hiyerarşisi söz konusudur. İkincisi, template metodun override edilememesi önemlidir. Bu yüzden soyut üst sınıflardaki bu somut metotların final tanımlanması gerekir. Aksi halde OpenDocument'taki yapıyı ince ince kurmanın bir anlamı kalmaz; biri gelip MyApplication alt sınıfında OpenDocument'ı override ederek bütün framework'ün çalışma biçimini bozabilir.

> **Hocanın vurgusu:** Bozulmasını istemediğimiz ana ayrıntıların yazıldığı metot, üst sınıfta final olarak kodlanmalıdır. Bu metot, somut alt sınıf için bir şablon (template) oluşturur; ara ayrıntılar soyut metotlar üzerinden serbest bırakılır ama ana çalışma biçimi bozulmaz.

Hoca, bu çözümü daha önce öğrenilenlerle belki kendiliğinden de bulunabileceğini, ama yine de literatüre kazandırılmış bir çözüm olduğunu ekledi.

3. Örnek Gerçekleme: Farklı Veritabanlarından Kayıt Çekme

3.1 Problem

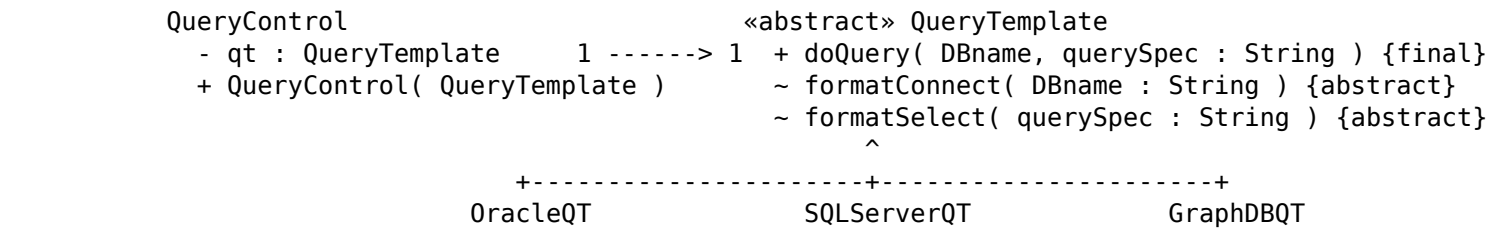
Örnek, Design Patterns Explained kitabından alınmıştır. Bir programda veritabanından kayıtlar çekilecektir; bu iş SQL ile, bir select komutu hazırlanarak yapılacaktır. Ancak uygulama farklı veritabanı yönetim sistemleriyle çalışabilmelidir. SQL komutları standart olsa da veritabanına bağlantı kurmak için yapılacak işler ve verilecek komutların ayrıntıları değişebilir. Özellikle stored procedure desteği isteniyorsa farklılıklar daha da artar.

Bütün bu farklılıklara rağmen değişmeyen temel adımlar vardır:

1. Bir bağlantı dizesi (connection string) oluştur.
2. Veritabanına bağlantı komutunu gönder.
3. Bir select komutu inşa et.
4. Bu komutu gönder.
5. Oluşan veri kümesini geri döndür.

3.2 Sınıf şeması

Slayttaki önerilen çözüm (ekrandan, 00:17:10):



Slayttaki notlar: komut biçimlendirme metotları iç çalışma ile ilgili görülerek protected tanımlanabilir; bir sınıf kütüphanesi veya çerçeve program hazırlanmadığına göre Java için paket görünürlüğü kullanılabilir. Şemada da bu metotlar ~ (paket) görünürlüğüyle gösterilmiştir. Hoca, paket görünürlüğünü bu metotların başka bir paketten çağrılıp çalışma bütünlüğünün bozulmasını önlemek için tercih ettiğini söyledi. Alt sınıflardan biri ilişkisel veritabanları dışında graf veritabanlarıyla da çalışabilecek bir çözüm önerir (GraphDBQT).

> **Ek bilgi (derste anlatılmadı):** Java'da paket görünürlüğündeki (erişim belirteci yazılmamış) bir soyut metot yalnız aynı paketteki alt sınıflarca override edilebilir. Başka paketteki bir alt sınıf bu metodu göremez ve gerçekleyemez; bu durumda sınıf soyut kalır. Bu yüzden paket görünürlüğü seçilirse OracleQT, SQLServerQT gibi somut sınıfların da QueryTemplate ile aynı pakette olması gerekir; farklı paketlerden genişletme istenirse protected uygundur.

3.3 İskelet kod

Hoca örneğin iskelet kodda çok fazla bir şey içermediğini, bu yüzden çalıştırılacak bir örnek olmadığını söyledi. Slayttaki kod (ekrandan, 00:18:30):

```
package templateMethod.example;
public abstract class QueryTemplate {
    public final void doQuery( String DBname,
                               String querySpec ) {
        String dbCommand;
        dbCommand = formatConnect( DBname );
        //dbCommand komutunu yürüterek bağlantıyı kur.
        dbCommand = formatSelect( querySpec );
        //dbCommand komutunu yürüterek kayıtları oku.
        //Oluşan veri kümesini geri döndür.
    }
    abstract String formatConnect( String DBname );
    abstract String formatSelect( String querySpec );
}
```

QueryTemplate soyuttur; doQuery hem final hem somuttur. Ana adımlar doQuery içinde sırasıyla yürütülür, değişen kısımlar soyut metotlar çağrılarak alt sınıflara bırakılır. Somut alt sınıflarda ise veritabanına özgü işlerin nasıl yapılacağı bu metotların gövdesine kodlanır. Slayt listesinde somut alt sınıfların kodları da (örneğin bir Oracle ve bir SQL Server gerçeklemesi ile bir `QueryControl` kullanım örneği) yer alıyordu; hoca bunlara ayrıntılı girmedi.

Hoca, Template Method'un "o kadar da zor bir kalıp olmadığını", ama akılda tutmanın yararlı olacağı bir çözüm verdiğini söyledi.

4. Template Method ile Strategy Karşılaştırması

Veritabanı problemini ilk görünce akla Strategy kalıbı gelebilir: bir iş yapılacak ve farklı veritabanları desteklenecek. Hoca, Strategy'nin üst türünün aslında bir arayüz (interface) olduğunu, orada genelde iç mantığın kodlanmadığını hatırlattı. Tek istisna, bir varsayılan algoritmanın işe yarayacağı durumdur; o zaman üst tür soyut, hatta somut sınıf olarak da kodlanabilirdi, ama bu nispeten nadir bir durumdur. Strategy kalıbında her somut strateji başlı başına bütün işi yapar.

Template Method'da ise bazı ayrıntılar sabittir, bazıları altta değiştirilebilir. Veritabanı örneğinde yapılacak iş ana hatlarıyla bellidir ama bazı ayrıntıları değişecektir; o zaman en mantıklısı değişmeyen ana işleri bir template metotta kodlamak, değişebilecek olanları üst sınıfta soyut bırakıp alt sınıflarda doldurmaktır.

> **Hocanın vurgusu:** "Böyle bir program hazırlanacak, hangi tasarım kalıbını seçerdiniz, neden?" türünden bir soruda Strategy seçilirse farklı veritabanlarıyla çalışma ihtiyacı karşılanır ama bu problemin yalnız yarısına hitap eder. Problemin bütününe hitap etmek için ortak yapılacak kısımların olduğunu görüp Template Method'u seçmek daha doğrudur.

5. Visitor Kalıbı: Amaç, Sorun ve Eleştiri

5.1 Amaç ve sorun

Hoca Visitor'ı "biraz karışık", "ağır sıklet" kalıplardan biri ve özel eleştirileri olan bir kalıp olarak tanıttı. Slayttaki tanım (ekrandan, 00:24:10):

- Amaç: herhangi bir nesne yapısını oluşturan elemanlar üzerinde yapılacak bazı işlemler tanımlamak.
- Sorun: nesne yapısı her türden bağıntılar içeren sınıfların örnekleri arasında kurulmuş olabilir; üzerinde işlem yapılan elemanların sınıf yapılarını değiştirmek istemiyoruz.

Nesne yapısı denince akla bir Composite gelebilir, ama Composite kalıbını kullanmak şart değildir; herhangi bir ağaç yapısı, hatta bir liste olabilir. Yine de daha çok ağaç düşünülür.

Hoca bu sorunun yanlış anlaşılmaması gerektiğini vurguladı: kaynak kod kapalı değildir, proje şu an sıfırdan yazılmaktadır ve bütün sınıflara bu aşamada biz karar veriyoruz. Amaç, zaman içinde yeni ihtiyaçlar ortaya çıktığında, parça-bütün yapısını oluşturan nesnelerin kaynak kodlarına yeni eklemeler yapmak zorunda kalmadan yeni işler yapabilmektir. Hoca bu hedefi "biraz fazla iddialı (ambitious)", belki "haddini aşan" bir istek olarak niteledi.

5.2 Eleştiri

Slayttaki eleştiri: madem daha yapılacak işlemler var, eylemler ve sorumluluklar tam olarak belirlenmemiş demektir; dolayısıyla mevcut sınıf yapılarını değiştirerek farklı bir tasarım yapmak daha doğru olabilir. Hoca, "yakın gelecekteki gereksinimlere göre tasarlarız, çok farklı gereksinimler gelirse projeye yeni bir faz olarak ilerleriz" diyenlerin belli ölçüde haklı olduğunu, eleştirinin de bu noktaya odaklandığını söyledi.

Buna karşın kalıbın mantıklı olduğu bir durum vardır: geriye doğru uyumluluğu çok uzun süre korumak istediğimiz bir framework hazırlıyorsak. Çok uzun zaman içinde çok farklı ek işlevler istenebilir ve bunların hepsini öngörmek mümkün değildir. Geriye doğru uyumluluk da uzun süre korunacaksa Visitor kullanmak mantıklıdır.

6. Visitor GoF Örneği: Derleyici Ağacı

6.1 Problem

Dörtlü Çete'nin örneği bir derleyicidir. Slayttaki maddeler: yazılan bir derleyici kaynak kodu farklı türlerden token'lara ayırıyor; bu parçalar (Node) farklı tiplerden oluşmaktadır; derleyici bütün parçalar üzerinde çeşitli işlemler yapacaktır; farklı tip parçalarda işlemler farklı yürütülecektir.

Hoca bunu bir örnekle çizdi: `int x = 3+5` komutu. Derleyici bu komutu token'larına ayırıp bir ağaç kurar:

```
int (tür bildirimi düğümü)
 └── x (değişken düğümü)
      └── = (atama düğümü)
           └── + (toplama düğümü)
                ├── 3 (sabit düğüm)
                └── 5 (sabit düğüm)
```

Önce tam sayı türünden bir değişken tanımlamayı sağlayan `int` bir token olur; onun etkilediği düğüm `x` değişkenidir; `x`'e bir atama yapılır; atamanın değeri bir toplama işleminin sonucudur; toplama ise 3 ve 5 sabit düğümleri üzerinde yürütülür.

6.2 Ağaç üzerindeki işlemler

Derleyici bu ağacı dolaşarak farklı işler yapar ve her iş her düğümü ilgilendirmez:

- Pretty printing: örneğin eşittirin iki yanına boşluk konmamışsa okunabilirliği iyileştirmek için düğümleri dolaşıp gerektiğinde boşluk ekler.
- Vurgulama (highlighting): yalnız türler koyu yazılacaksa ağaçta yalnız `int` düğümü koyu yapılır; sabit değer, atama ve aritmetik işlem düğümlerinde bir şey yapılmaz, bunlar atlanır.
- Hata ayıklama (debugging): düğümler dolaşılırken `x` değişkeninin değeri izlenmek isteniyorsa yalnız değişken düğümleri dolaşılır.
- Tür denetimi (type checking): özellikle atamada, eşitliğin solundaki düğümün türü ile sağdaki ifadenin sonucunun veri türü uyuşuyor mu diye bakılır.
- Kod üretimi (code generation).

6.3 Kalıpsız çözüm ve sorunu

Kalıp kullanılmazsa bu işlerin her biri Node hiyerarşisinde bir metot olur (type checking, code generation, pretty printing vb.) ve değişken referansı düğümü, atama düğümü gibi bir dolu kardeş alt sınıfın her birinde kodlanır. Ama her düğümde her işin anlamı yoktur. Örneğin

değişken referansı düğümünde pretty printing gerekmiyorsa bile kalıtım ya da arayüz kuralı gereği o metodu, boş gövdeli bile olsa, kodlamak zorunda kalırız. Hoca bunu bir zorlama olarak niteledi.

6.4 Visitor çözümü

Slayttaki çözüm: derleyici parçaları tek tek ziyaret ederek her parça üzerinde sadece gerekli işlemleri yapar (ekrandan, 00:30:00). Şema:

```
NodeVisitor (soyut)
  VisitAssignment(AssignmentNode)
  VisitVariableRef(VariableRefNode)
      ^
      +-- TypeCheckingVisitor   (VisitAssignment, VisitVariableRef)
      +-- CodeGeneratingVisitor (VisitAssignment, VisitVariableRef)

Program <>----> Node (soyut)
                  Accept(NodeVisitor)
                    ^
                    +-- AssignmentNode   Accept(NodeVisitor v) { v->VisitAssignment(this) }
                    +-- VariableRefNode  Accept(NodeVisitor v) { v->VisitVariableRef(this) }
```

Düğüm yapısı bir öncekine benzer kalır; yapılması gereken işler hâlâ vardır, ama artık ayrı bir ziyaretçi yapısında toplanır. Üstte soyut bir NodeVisitor arayüzü bulunur; bu aşamada yalnız dolaşma altyapısı kurulmaktadır. NodeVisitor hangi tür düğümleri ziyaret edebileceğini (atama, değişken referansı vb.) metotlarıyla belirtir. Somut alt sınıflarında yalnız tür denetimi için dolaşan bir ziyaretçi, kod üretmek için dolaşan bir başka ziyaretçi vardır. Yani önceki düğüm yapısındaki her iş için ayrı bir somut ziyaretçi tanımlanır.

Ziyaretçilerin parça-bütün yapısını dolaşabilmesi için düğüm yapısına bir `Accept` metodu eklenir. Bu metot parametre olarak soyut ziyaretçi türünden bir nesne alır ve her somut düğüm türünde gövdesi ayrıca tanımlanır. Gövdede o düğümün nasıl ziyaret edilmesi gerektiği kodlanır: atama düğümü "ben bir assignment düğümüyüm, beni assignment olarak ziyaret et" der ve ziyaretçinin genel bir visit metodunu değil `VisitAssignment` metodunu çağırır. Örneğin bir TypeCheckingVisitor, atama düğümünde yapması gereken işleri `VisitAssignment` içine kodladığı için düğüm ziyaretçiyi kabul edip "beni böyle ziyaret edeceksin" dediğinde ziyaretçi "daveti icabet eder", o işi yapar ve başka bir şey yapmaz.

Slayttaki maddeler: düğüm gerçeklemeleri, kendi üzerlerindeki gerekli işlemleri çağıracak ziyaretçi gerçeklemelerini Accept metodu ile kabul eder; bu şekilde parçalarda sadece gerekli işlemler yürütülür; çözümün zayıf yönü, düğümlerin ziyaretçinin hangi metodunun kendi düğüm türleri için olduğunu bilmesidir.

> **Ek bilgi (derste anlatılmadı):** Accept ile Visit metotlarının bu iş birliği literatürde "çift gönderim" (double dispatch) diye anılır. Hangi kodun çalışacağı iki nesnenin çalışma anındaki türüne birlikte bağlıdır: Accept çağrısı düğümün türüne göre, içindeki Visit çağrısı ise ziyaretçinin türüne göre seçilir. Java ve C++ gibi diller yalnız tek gönderim (çağrılan nesnenin türüne göre seçim) yaptığı için Visitor bu ikinci seçimi elle kurar.

6.5 Hocanın eleştirileri

Hoca, somut düğüm sınıflarının ziyaretçi arayüzünü bütünüyle bilmek zorunda olduğunu, bunun bilgi açısından sorun olmasa da arada bir bağlaşım yarattığını söyledi: her düğüm, ziyaretçi arayüzündeki hangi metot üzerinden ziyaret edilmesi gerektiğini bilmek zorundadır. Hoca bunu küçük bir eleştiri olarak dile getirdi.

Hoca ayrıca yaklaşımı "kulağı tersten tutmaya" benzetti: normalde ne yapılacağı bilinir ve tasarım ona göre yapılır; burada ise farklı yapılacak işler ortaya çıktıkça ona göre kodlama yapılabilecek bir altyapı kurulur. Yeni bir ziyaretçi eklemek kolaydır, ama yepyeni bir iş türü düğümlerde yeni bir ziyaret biçimi gerektiriyorsa üst ziyaretçi arayüzüne yeni bir metot eklenmesi ve ilgili düğümlerde bu metodun çağrılması gerekebilir. Bu da bakımı zorlaştıran ek bir karmaşıklıktır. Hocanın özetiyle Visitor "artısı da eksisi de olan bir kalıp"tır.

7. Visitor Kalıp Yapısı ve Zayıf Yönleri

7.1 Kalıp yapısı

Slayt 89'daki kalıp yapısı (ekrandan, 01:50:00):

```
Client ----> Visitor (soyut)
               VisitConcreteElementA(ConcreteElementA)
               VisitConcreteElementB(ConcreteElementB)
                 ^
                 +-- ConcreteVisitor1 (VisitConcreteElementA, VisitConcreteElementB)
                 +-- ConcreteVisitor2 (VisitConcreteElementA, VisitConcreteElementB)

Client ----> ObjectStructure <>----> Element (soyut)
                                       Accept(Visitor)
                                         ^
                                         +-- ConcreteElementA
                                         |     Accept(Visitor v) { v->VisitConcreteElementA(th
                                         |     OperationA()
                                         +-- ConcreteElementB
                                               Accept(Visitor v) { v->VisitConcreteElementB(th
                                               OperationB()
```

Bir nesne yapısı ve onu oluşturan çeşitli elemanlar vardır. Hoca, elemanların ortak bir arayüzünün olmasının yararlı olacağını, zaten çoğu zaman olacağını söyledi: veri yapısının işlemlerinin tanımlandığı bir üst sınıf zaten vardır. Yapılması gereken, bu üst sınıfa bir `Accept` metodu eklemek, ona parametre olarak ziyaretçi arayüzü türünden bir nesne vermek ve var olan somut elemanların içine Accept'in gövdesini kodlamaktır. Eleman, ziyaretçiye "hoş geldin, seni kabul ediyorum, ama benim üzerimde yalnız şu işlemleri yapabilirsin" der.

7.2 Kalıp bileşenleri

Slayt 90'daki bileşenler (ekrandan, 00:36:50):

- ObjectStructure: elemanları dolaşılacak yapı.
- Element: dolaşılacak elemanların ortak arayüzü. Accept metodu Element arayüzünde tanımlanır ve ziyaretçiler bu metotla kabul edilir.
- ConcreteElementX: dolaşılacak elemanların gerçeklemeleri. Ziyaretçilerin hangi metodunun bu tür eleman için olduğu bilgisine sahip olmalıdır.
- Visitor: ziyaretçilerin ortak arayüzü. Her tür eleman için olan ziyaret işlemlerinin tamamı burada tanımlanmak zorundadır.
- ConcreteVisitorX: ziyaretçi gerçeklemeleri. İş yapılırken elemanlar ayrı ayrı ziyaret edileceğinden, ziyaretler arasında durum bilgisinin saklanması için gerekli kodlama yapılmalıdır.

7.3 Zayıf yön: yeni eleman türü eklemek

Slayttaki zayıf yön: yeni ConcreteElement türleri oluşturmak zordur; her yeni tip için Visitor arayüzüne ve gerçeklemelerine yeni bir metot eklemek gerekecektir. Hoca bunu şöyle açtı: ConcreteElementC eklenirse Visitor arayüzüne `VisitConcreteElementC` eklenir ve mevcut somut ziyaretçilerin bu yeni elemanla bir işi olmasa bile bu metodu boş gövdeyle kodlamaları gerekir. Bu, kalıbı ilk anlatırken söylenen dezavantajın tekrarıdır.

Bu yüzden Visitor, yapısı oturmuş bir parça-bütün ilişkisi üzerinde kurulmalıdır. İlk anlatılan motivasyon tamamen geçerliyse, yani parça-bütün ilişkisindeki parçaların kendisi değişmiyor ve yalnız onlarla yapılacak işler değişiyorsa kalıbı uygulamak mantıklıdır.

8. Örnek Gerçekleme: Proje Yönetim Yazılımı, Çözüm 1

8.1 Problem

Örnek Applied Java Patterns kitabından alınmıştır (ekrandan, 00:40:00). Hoca önce başka bir olası örnek de verdi: Composite örneğindeki makine ve makine parçaları üretilen fabrikada

parçalar üzerinde çeşitli kalite kontrol işlemleri yapılacaksa Composite örneği Visitor ile genişletilebilir; böylece aynı problemde iki kalıp birlikte gerçeklenir. Hoca bunu gönüllü bir alıştırma olarak önerdi.

Asıl örnekte bir proje yönetim yazılımı hazırlanmaktadır. Temel varlıklar:

- Project: proje hiyerarşisinin kökü.
- Task: projede tanımlı bir görev.
- Deliverable: projenin ara çıktıları veya sonucu olarak üretilebilecek herhangi bir kod, belge, ürün vb.

8.2 Sınıf şeması

Hoca, henüz hiçbir kalıp düşünmeden makul bir nesneye dayalı tasarımın şöyle olabileceğini gösterdi (ekrandan, 00:46:00):

```
«Abstract» ProjectItem
  - name, description : String
  + ProjectItem(String, String)
  + getName() : String, getDescription() : String
  + setName(String), setDescription(String)
      ^
      +-- Deliverable
      |      - materialsCost, productionCost : double
      |      + Deliverable(String, String, Contact, double, double)
      |      + getOwner() : Contact, getMaterialsCost() : double, getProductionCost() : double
      |      + setOwner(Contact), setMaterialsCost(double), setProductionCost(double)
      +-- Task
      |      - projectItems : ArrayList<ProjectItem>
      |      - timeRequired : double
      |      + Task(String, String, Contact, double)
      |      + getProjectItems(), addProjectItem(ProjectItem), removeProjectItem(ProjectItem)
      |      + getOwner() : Contact, getTimeRequired() : double
      |      + setOwner(Contact), setTimeRequired(double)
      +-- Project
             - projectItems : ArrayList<ProjectItem>
             + Project(String, String)
             + getProjectItems(), addProjectItem(ProjectItem), removeProjectItem(ProjectItem)

Project <>----> ProjectItem (bileşim)
Task --owner 1--> Contact,  Deliverable --owner 1--> Contact

ProjectCalculator  (Project ile 1-1)
  + calculateTimeRequired() : double
  - calculateTimeRequired( subTask : Task ) : double
  + calculateCost() : double
  - calculateCost( subTask : Task ) : double

Report (Project ile 1-1, ProjectCalculator'ı kullanır)
  + Report()
  + main()
```

Proje, çeşitli iş kalemlerinden oluşur; bu yüzden bir Project ve bir ProjectItem sınıfı vardır. Bir iş kalemi ya görev (Task) ya da çıktı (Deliverable) olacağından ProjectItem soyuttur. Ortak bilgiler ad ve tanımdır; hoca bir kimlik (ID) de eklenebileceğini ama örnekte bununla uğraşılmadığını söyledi.

Hoca, Project tarafında aslında bir Composite yapısı görüldüğünü belirtti: parça-bütün ilişkisi ve kalıtım ilişkisi sayesinde proje ara düğüm, görev ve çıktılar çocuk düğümlerdir. Proje iş kalemlerini bir ArrayList'te tutar; hoca bunu bir elemana hızlı ulaşmak için seçildiğini düşündü. Görevin de alt görevleri ve çıktıları olabilir; `projectItems` listesinin ProjectItem türünden olması buna izin verir.

> **Hocanın vurgusu:** Bu yapıda çevrimsel bağımlılık kurmamak gerekir. Bir görevin altına projenin kendisi verilirse çevrim oluşur ve sonsuz döngüye girilebilecek senaryolar çok rahat çıkabilir.

Project'te kurucu, öğe ekle ve öğe sil metotları vardır; Task'ın da alt öğeleri olabileceği için onda da `addProjectItem` ve `removeProjectItem` bulunur; Deliverable'da ise böyle bir şey yoktur. Görevin alt görevleri de olabiliyorsa bu tam bir Composite'tir.

Hoca şemadaki bir eksiği de düzeltti: Project'ten ProjectItem'a parça-bütün ilişkisi gösterildiği halde Task'tan ProjectItem'a gösterilmemişti; bunu bir elmasla çizdi. Ancak görevin alt görevleri olamayacaksa ve yalnız sıfır ya da daha fazla çıktısı olacaksa, listeyi ProjectItem değil Deliverable türünden tanımlamak ve parça-bütün ilişkisini Deliverable'a kurmak daha doğru olur. İki elmastan biri seçilmelidir.

Görevin bir sahibi (owner) vardır: işi yapacak kişi, Contact sınıfından bir nesne. Çıktının da bir sahibi olabilir; ikisi de bire bir sahipliktir. Görev için gereken zaman `timeRequired` üyesinde tutulur; hoca buna istenirse maliyet gibi başka bilgiler de eklenebileceğini söyledi. Çıktının malzeme bedeli ve üretim bedeli üye alan olarak tanımlanmıştır. Kurucular gereken parametreleri alır, üst sınıfın kurucusunu çağırır; getter ve setter metotları da vardır.

Proje yöneticisi olarak projenin ne kadar zamana ihtiyaç duyduğunu ve maliyetini bilmek, takip etmek ve üst makamlara rapor vermek gerekir. Bunun için ProjectCalculator sınıfı hazırlanmıştır. Bir düğümün alt düğümleri olabileceği için her public hesap metodu, alt görevleri de dolaşan ilgili private metodu çağırır. Hepsini yürütmek için main metodu olan bir Report sınıfı vardır. Hocanın değerlendirmesine göre bu tasarım problemi çözer; ihtiyaçlar ve rapor belliyse bu yapıyla gitmenin bir sakıncası yoktur.

8.3 Ne zaman Visitor düşünülür

Hoca, proje yönetim yazılımı zaman içinde başka şekillerde genişletilmek isteniyorsa durumun değiştiğini söyledi. Örneğin süre ve maliyet hesaplarına ek olarak ileride "en maliyetli iş kalemini bul" ya da "en uzun süren iş kalemini bul" gibi sorgular gelebilir. Bu versiyonda ProjectCalculator'a yeni metotlar eklenerek bunlar da yapılabilir; ama her seferinde parça-bütün yapısı üzerinde tekrar tekrar dolaşma kodu yazmak gerekir ve hesap metotları birlikte incelendiğinde, en azından yapıyı dolaşmak amacıyla, içlerinde çok ortak kod olduğu görülür.

8.4 Çözüm 1'in kodu

Hoca örnek kodda bir yüksek lisans eğitimini proje olarak modellediğini gösterdi (ekrandan, 01:00:00). Proje "Yüksek Lisans: Lisansüstü eğitim" adını taşır. Görevler dönemlerdir (örneğin bir dönem 4 ders, 15 hafta; tez dönemi 30 hafta), çıktılar ise dersler, tez metni ve bir bildiri ya da makaledir. Hoca bunun eski kurallara göre olduğunu ve yalnız örnek olarak düşünülmesi gerektiğini, Contact'ın örnekte önemli olmadığı için `null` geçildiğini söyledi. Deliverable kurucusundaki iki maliyet değeri bu örnekte şöyle yorumlanmıştır: malzeme maliyeti derse devam için harcanan saat (haftada 3 saat × 15 hafta = 45 saat), üretim maliyeti sınavlara çalışmak için harcanan saat. Ekrandaki satırlardan bazıları:

```
urun = new Deliverable("BLM 5130", "Nesneye Dayalı Tasarım ve Modelleme", null, 45, 40);
gorev.addProjectItem(urun);
...
project.addProjectItem(gorev);
gorev = new Task("2014-2 dersleri","2014-2015 Bahar yarıyılı (4 ders)",null,15);
...
gorev = new Task("BLM 5198","2015-2016 Yüksek Lisans",null,30);
urun = new Deliverable("Tez metni", "Bez ciltli tez", null, 90, 270);
gorev.addProjectItem(urun);
```

ProjectCalculator'ın süre hesabı (ekrandan, 01:00:40):

```
public class ProjectCalculator {
    private Project project;
    public ProjectCalculator(Project project) {
        this.project = project;
```

```
    }
    public double calculateTimeRequired( ) {
        double time = 0.0;
        for( ProjectItem item : project.getProjectItems() ) {
            if( item instanceof Task ) {
                time += calculateTimeRequired((Task)item);
            }
        }
        return time;
    }
    private double calculateTimeRequired( Task subTask ) {
        double time = subTask.getTimeRequired();
        for( ProjectItem item : subTask.getProjectItems() ) {
            if( item instanceof Task ) {
                time += calculateTimeRequired((Task)item);
            }
        }
        return time;
    }
    // calculateCost() ve calculateCost(Task) benzer biçimdedir.
}
```

Hoca burada kalıbın asıl avantajının koddan görüldüğünü söyledi. Tasarımda çıktıların zaman açısından bir masrafı yoktur, görevlerin vardır. Zaman hesaplanırken çıktılara, maliyet hesaplanırken de görev düğümlerinin kendisine bakmanın anlamı yoktur (görevlerin içindeki çıktılara ulaşmak için dolaşılır, ama görev düğümünde maliyet hesaplanmaz). Bu yüzden hiyerarşideki her düğüm ziyaret edildiğinde "bu bir görev düğümü mü, yoksa çıktı mı" diye tek tek bakıp ona göre davranan kod yazmak gerekir.

> **Hocanın vurgusu:** "Bu nesne şu türden mi? Evet. Öyleyse onu o türe çevir, sonra orada yapılması gereken işleri yap" türünden kod yazmak nesneye dayalılığın çok güzel bir kullanımı değildir ve kodun esnekliğini çok azaltır. Bu çözüm, bir "hızlı ve kirli düzeltme" ya da "gözümü kapatırım, vazifemi yaparım" türünden bir çözümdür.

8.5 Çalıştırma ve Eclipse notu

Hoca kodu çalıştırırken Eclipse'te Türkçe karakterlerin bozuk göründüğünü düzeltti ve proje uyumluluğuyla ilgili bir hata aldı. Çözüm olarak yeni bir proje (NDTM-denemeler) içinde bir paket (hafta07a) açıp örnek paketteki sınıfları kopyalayıp yapıştırdı ve kod çalıştı. Hoca, ders notu kaynak kodlarını Eclipse'e içe aktarırken benzer bir uyumluluk sorunuyla karşılaşılırsa sıfırdan yeni bir proje oluşturup bir paket açarak ilgili örnek paketteki dosyaları buraya yapıştırmanın alternatif bir çözüm olduğunu söyledi. Sorunun nedenini kesin bilmediğini, kurduğu kalite ve metrik eklentileri ya da farklı sürüm hedefleriyle ilgili olabileceğini ekledi.

Program çıktısı (ekrandan, 01:10:00):

```
Proje adı ve açıklaması: Yüksek Lisans: Lisansüstü eğitim (Yüksek lisans)
Proje süresi: 60.0 hafta.
Proje maliyeti: 1174.0 saat çalışma.
```

9. Örnek Gerçekleme: Proje Yönetim Yazılımı, Çözüm 2 (Visitor)

9.1 Sınıf şeması

Hoca, önceki şema zaten karmaşık olduğundan ikinci şemayı yalnız Visitor kalıbını kurmak için yapılan eklemeleri gösterecek şekilde sadeleştirdi; önceki çözümle aynı olan üyeler gösterilmemiştir (ekrandan, 00:50:00):

```
«Abstract» ProjectItem
  - name, description : String
  + accept( ProjectVisitor ) {abstract}
      ^
```

```
      +-- Deliverable  (- materialsCost, productionCost : double;  + accept(ProjectVisitor))
      +-- Task         (- projectItems : ArrayList<ProjectItem>; - timeRequired : double;
      |                 + accept(ProjectVisitor))
      +-- Project      (- projectItems : ArrayList<ProjectItem>;  + accept(ProjectVisitor))

Project ◆----> ProjectItem
Deliverable --1--> Contact,  Task --1--> Contact
Deliverable, Task, Project - - -> ProjectVisitor (bağımlılık)

«interface» ProjectVisitor
  + visitDependentTask(DependentTask)
  + visitDeliverable(Deliverable)
  + visitTask(Task)
  + visitProject(Project)
      ^ (gerçekleme)
      +-- ProjectCostVisitor
      +-- ProjectTimeVisitor

Report (+ Report(), + main())  - - -> ProjectCostVisitor, ProjectTimeVisitor
```

ProjectItem'a kalıbın önerdiği şekilde, parametre olarak ProjectVisitor alan soyut bir `accept` metodu eklenir. Üstte soyut tanımlandığı için Project, Task ve Deliverable'da somut olarak kodlanması gerekir. Ziyaretçi arayüzünde her düğüm türü için, o türden parametre alan bir `visit...` metodu vardır.

Hoca şemada fazladan bir `visitDependentTask` metodu olduğunu fark etti. Örnekte DependentTask diye bir sınıf da vardı; hoca kalıbın örneklenmesine hiçbir katkısı olmadığı için onu şemanın dışında bırakmış, ama metodu arayüzden silmemişti. Bu metodu da ProjectVisitor arayüzünden silmeyi not aldı.

Report sınıfı artık maliyeti hesaplayan ziyaretçiyi ve zamanı hesaplayan ziyaretçiyi ayrı ayrı çalıştırır. Önceki çözümde bunlar ProjectCalculator içinde ayrı metotlardı; şimdi "her farklı sorumluluk farklı bir sınıf olsun" diyerek ayrı sınıflara alınmıştır.

9.2 Kod

Kod yeni bir pakette (hafta07b) çalıştırıldı. Project'in `accept` metodu ziyaretçiye "ben bir projeyim, benim üzerimde visitProject işini yürüteceksin" der; Task'ınki "ben bir task nesnesiyim, benim üzerimde kendi visitTask işlemini yürütebilirsin, ancak buna izin veriyorum" der.

ProjectCostVisitor'ın başı (ekrandan, 01:14:40):

```java
public class ProjectCostVisitor implements ProjectVisitor {
    private double cost = 0.0;

    public ProjectCostVisitor(Project project) {
        project.accept(this);
    }

    public void visitDependentTask(DependentTask p) {
        visitTask(p);
    }

    public void visitDeliverable(Deliverable p) {
        ...
```

Report'un main metodu (ekrandan, 01:12:30):

```java
Report rep = new Report();
ProjectCostVisitor vc = new ProjectCostVisitor(rep.project);
System.out.println("Proje adı ve açıklaması: " + rep.project.getName() + ...);
System.out.println("Proje maliyeti: " + vc.getCost() + " saat çalışma.");
```

```
ProjectTimeVisitor vt = new ProjectTimeVisitor(rep.project);
System.out.println("Proje süresi: " + vt.getTime() + " hafta.");
```

Ziyaretçinin kurucusuna hiyerarşinin kök düğümü olacak nesne verilir ve kurucu, bu kök düğüme `accept(this)` diyerek dolaşma zincirini başlatır. Hoca, dolaşmanın ziyaretçi oluşturulur oluşturulmaz değil, bir kontrolcünün daha sonra başlatmasıyla (örneğin kullanıcı bir düğmeye basınca) da yapılabileceğini, bunun için çağrının kurucudan ayrı bir metoda alınabileceğini söyledi.

ProjectTimeVisitor da benzer şekilde projeyi alır. Maliyet ziyaretçisi bütün düğümleri dolaşmak zorundadır: görevse alt öğeleri arasında dolaşır, çıktıysa maliyetini hesaplar. Ama ProjectCostVisitor'da hiç `instanceof` yoktur; tür denetimi yapmak zorunda kalınan bir durum yoktur. Hangi tür düğümün ziyaret edileceği metot parametresinin türüyle söylenmiş olur.

Hoca bunun ilk başta bir dezavantaj olarak eleştirildiğini hatırlattı: düğüm türleriyle ziyaretçinin metotları arasında fazladan bir bağlaşım oluşuyordu. Şimdi ise bu bağlaşım sayesinde instanceof yazmak zorunda kalınmıyor. Hoca bu takası "kırk katır mı kırk satır mı" kadar ağır görmediğini, ikisinin de idare edilebilir olduğunu söyledi. Çalıştırınca yine 60 hafta ve 1174 saat hesaplandı.

9.3 Kod tekrarı karşılaştırması

Kalıbın başında yinelenen kodun (duplicate code) azaltılabileceği söylenmişti; hoca iki çözümü bu açıdan karşılaştırdı. ProjectCalculator'da "instanceof şuysa bunu yap" kalıbı tekrar tekrar geçer ve metotların gövdeleri birbirine benzer. Ziyaretçilerde de bir benzerlik görülebilir, ama bunlar tek elemanlı, tek satırlık döngülerdir; benzerlik varsa bile boyutu daha azdır. Report sınıfında nesneleri oluşturup ekleyen kodda zaten iki çözümde de çok tekrar yoktu. Sonuçta kod tekrarı da biraz azalmıştır.

> **Hocanın vurgusu:** Bu kadar çabaya değip değmediği, kalıbın getirisinin projede ne kadar önemli olduğuna bağlıdır. Şirket olarak tutturulması gereken kalite hedefleri varsa bu çabaya girilebilir; ama kalite hedefleri amaç değil araç olmalıdır. Aksi halde birkaç metriği iyileştireyim derken süreçlerde daha büyük verimsizlikler ortaya çıkabilir.

10. Mediator Kalıbı

10.1 Amaç

Mediator, dönemde birlikte incelenen son kalıptır. Hoca kalıbın adını "ara bulucu", "yönlendirici" diye karşıladı ve bunun da bir noktada eleştirilecek, "bütün problemi halının altına süpürmek" gibi davranan bir kalıp olduğunu söyledi.

Slayttaki amaç (ekrandan, 01:33:15):

- Bir nesneler kümesinin aralarındaki etkileşimlerin sarmalanması.
- Böylece nesneler birbirleri ile doğrudan değil de dolaylı olarak ilişki kurarlar.
- Sonuç olarak söz konusu nesnelerin arasındaki ilişkiler bu nesneleri değiştirmeden de değiştirilebilir.

Dolaylı ilişkiyi sağlayan nesne, kalıba adını veren Mediator sınıfından bir nesnedir. Etkileşimde bulunan nesnelerin kaynak kodları değiştirilmeden yalnız bu nesne değiştirilerek aralarındaki ilişkiler değiştirilebilir.

10.2 GoF örneği: font seçme penceresi

Slayttaki örnek: çeşitli GUI bileşenleri içeren bir font seçme penceresi hazırlanıyor; GUI bileşenleri kullanılarak font ile ilgili yapılan her değişiklik anında örnek metin alanına yansıtılacaktır (ekrandan, 01:34:00). Pencerenin en üstünde önizleme alanı ("The quick brown fox…") vardır; altında font ailesi giriş alanı ve listesi, kalınlık (Weight: medium, bold, demibold) ve eğim (Slant: roman, italic, oblique) radyo düğmeleri, boyut alanı ve "condensed" kutusu, Cancel ve OK düğmeleri bulunur.

Nesne şemasında aClient, aListBox, aButton ve anEntryField nesnelerinin her biri bir `director` referansıyla aFontDialogDirector nesnesine bağlıdır; aFontDialogDirector da bu bileşenleri tanır.

Kalıp olmadan bileşenler birbirleriyle doğrudan ilişkilendirilebilir. Hepsi önizleme alanıyla ilişkilidir, ama bunun ötesinde kendi aralarında da ilişkiler vardır: örneğin radyo düğmelerinde birine tıklanınca diğeri boşaltılmalıdır, roman ile italic de kendi arasında ilişkilidir. Bunlar için "bunun tıklandığını gör, ötekini boşalt" gibi epey kod yazmak gerekir. Bileşenlerin hepsini birbiriyle konuşturmak yerine ortak bir font director nesnesi üzerinden konuşturmak yarar sağlar.

10.3 Örnek çözüm

Slayttaki örnek çözüm (ekrandan, 01:38:30): GUI nesnelerine ortak bir arayüz hazırlanarak aralarındaki etkileşimleri genel olarak tanımlayacak bir başka soyut sınıf tasarlanır; bu soyut sınıftan gerçeklemeler oluşturmak için Template Method kalıbı kullanılır.

```
DialogDirector (soyut)   <---director---  Widget (soyut)
  ShowDialog()                              Changed()  -> director->WidgetChanged(this)
  CreateWidgets()        (soyut)              ^
  WidgetChanged(Widget)  (soyut)              +-- ListBox     GetSelection()
      ^                                       +-- EntryField  SetText()
      |
FontDialogDirector
  CreateWidgets()
  WidgetChanged(Widget)
  --list-->  ListBox
  --field--> EntryField
```

Hoca, GUI nesnelerinin hangi GUI framework'ü ile hazırlandıysa orada zaten bir kalıtım hiyerarşisinin bulunduğunu söyledi: soyut bir Widget sınıfının somut alt sınıfları olarak liste kutuları, giriş alanları, radyo düğmeleri vb. Olay yakalama da framework'te ya Observer türünden listener'larla ya da Command yapısıyla zaten kuruludur. Bize düşen, varsa change metodunu kullanmak, yoksa Widget'a bir change metodu eklemektir.

Font değişikliği penceresine özgü FontDialogDirector, hangi bileşenin değiştiğini algılar; örneğin liste kutusu değişmişse onun seçimini alır ve oradaki font değerini önizleme kutusuna uygular. `ShowDialog` GUI framework'ünde zaten olan bir metottur; DialogDirector sınıfı da framework'ün bir parçası olabilir de olmayabilir de, bu çok önemli değildir. Önemli olan, GUI bileşenlerini artık birbirleriyle değil font director ile ilişkilendirmektir. Director ya ilgili bileşenleri sorgulayarak ya da bileşenin "ben değiştim" bildirimini alarak ortaya çıkması gereken yeni durumu yürütür.

> **Ek bilgi (derste anlatılmadı):** Mediator ile Observer sıkça birlikte kullanılır: bileşenler (colleague) değişikliklerini mediator'a bir olay bildirimiyle iletebilir, mediator da onları gözleyen taraf olur. Fark amaçtadır: Observer bir nesnedeki değişikliği bilinmeyen sayıda ilgiliye duyurmayı, Mediator ise belirli bir nesne grubunun karşılıklı etkileşim mantığını tek bir yerde toplamayı hedefler.

10.4 Kalıp yapısı

Slayt 110'daki yapı (ekrandan, 01:40:00): Colleague gerçeklemeleri kendi aralarında doğrudan değil, bir Mediator gerçeklemesi üzerinden iletişim kurarlar.

```
Mediator (soyut) <----mediator---- Colleague (soyut)
    ^                                   ^
    |                        +---------+---------+
ConcreteMediator ------> ConcreteColleague1   ConcreteColleague2
        |                                          ^
        +------------------------------------------+
```

Normalde birbirleriyle etkileşimde olacak meslektaşlar (colleague; "eşler" diye de düşünülebilir) kendi aralarında ilişkide olmaz, hepsi aynı ara bulucuyla ilişkilidir. Soyut Mediator sınıfı

yapılması gereken işlerin arayüzlerini içerir; somut ConcreteMediator bu metotları doldurup gereken işleri yapar. Üst sınıfa varsayılan davranışlar eklenebilir; kalıtımın getirdiği her türlü olanaktan yararlanılabilir. Kalıp "illa şöyle yapın" demez; tek önerisi, N^2 ilişkiyi araya bir $(N+1)$'inci bileşen koyarak N etkileşime indirmektir ve bunun için karmaşık olmayan bir altyapı önerir.

10.5 Bağlantı sayısı ve eleştiri

Hoca, kalıp kurulmadan önce bileşenlerin birbirini görüp birbiriyle konuşacağını, beş bileşen için "5 kere 5, 25" bağlantı olabileceğini, mediator konunca ise yalnız 5 ilişki kalacağını söyledi; ardından bağlantıların tek yönlü mü çift yönlü mü sayıldığından emin olamadı ve "nasıl sayıyorsak sayalım, daha az bağlaşım olacak" dedi.

> **Ders sonrası not:** Hoca 5 nesne arasındaki doğrudan bağlantıyı "5 kere 5, 25" diye saydı. Bir nesnenin kendisiyle bağlantısı sayılmazsa 5 nesne arasında $5 \cdot 4/2 = 10$ yönsüz (ya da $5 \cdot 4 = 20$ yönlü) bağlantı olabilir. "N^2" ifadesi bu sayının N ile karesel büyüdüğünü anlatır; mediator ile bağlantı sayısı $N = 5$'e iner.

Hoca hemen ardından önemli bir düzeltme yaptı: bağlaşım (coupling) ilişkide olan sınıf sayısıyla ölçülür, nesne sayısıyla değil. Bu açıdan bağlaşım değişmez; kalıp yalnız ilişkide bulunan nesne sayısını azaltarak kod yazmayı kolaylaştırır. Bu kolaylık da N adet nesnenin birbiriyle konuşmasından doğan kod zorluğunu ConcreteMediator'da toplayarak sağlanır.

Mediator yokken belki 25 etkileşim vardı, ama bu etkileşimler çok karmaşık kodlar içermiyor olabilir. Karmaşık kodlar içeriyorlarsa etkileşim sayısını düşürmek çok yarar sağlar. Öbür türlü yapılan tek şey, nesnelerin kendi aralarında konuşmasından doğan karmaşıklığı ConcreteMediator'da toplamaktır ve o sınıfın kodu epey uzun olur.

> **Hocanın vurgusu:** Mediator'ların çoğu zaman Large Class kod kusuruna sahip olduğunu göreceğiz. Daha önce karmaşıklık colleague'ların kodundaydı; şimdi Mediator'ın koduna aktarılmış olur. Kalıp, uygulamaya değecekse kullanılır, değmeyecekse kullanılmaz.

Hoca bu kalıp için ayrıca bir örnek yapmaya ihtiyaç olmadığını söyledi; böylece tasarım kalıpları bitirilmiş oldu.

11. Kalıp Şemalarında UML Gösterimi

Dersin sonunda hoca, kalıp çözümlerinin sınıf şemalarında UML notasyonuna uyulması gerektiğini, dersin ön koşulunun nesneye dayalı tasarım ve UML bilgisi olduğunu hatırlattı ve örnek problem çözümlerindeki UML notasyonuna bakılmasını önerdi. Vurguladığı noktalar:

- Kalıtım oku: UML'deki doğru biçim içi boş üçgen uçlu oktur. Dörtlü Çete'nin kitabı UML önerilmeden önce yazıldığı için kalıtımı farklı bir biçimde çizer; hoca o eski biçimde çizilmesini görmek istemediğini söyledi.
- Soyut sınıf ya da arayüz: kitapta italik yazılarak gösterilir. Ama el yazısında italik mi düz mü olduğu anlaşılamaz; belki birinin yazısı doğal olarak yatıktır. Bu yüzden soyut sınıf UML kuralları gereği `<<Abstract>>` stereotipiyle işaretlenmelidir (hoca bunu slayttaki Visitor sınıfının üstüne iki küçüktür işaretiyle yazdı; ekrandan, 01:50:00). Arayüz için de `<<interface>>` kullanılır.
- Metotlar: geri dönüş değeri varsa iki noktadan sonra dönüş türü yazılır. Metot soyutsa ya da final ise küme parantezi içinde `{abstract}` veya `{final}` ile gösterilir (ekrandan, 01:50:00).
- Template Method'da final metot kalıbın çok önemli bir bileşenidir; şemada final olarak işaretlenmiş olması beklenir.
- Çizilen şemadan hem problem çözümü için hem de kalıbın ana sınıf yapısı için gerekli bütün parçalar görülebilmeli ve genel olarak nesneye dayalı tasarımın ana kurallarına uyulmalıdır.

Örnek Sorular

Soru 1 (derste sözlü, 00:15:14): Bir programda farklı veritabanı yönetim sistemlerinden SQL ile kayıt çekilecektir. Bağlantı dizesi oluşturma, bağlantı komutunu gönderme, select komutunu inşa etme, komutu gönderme ve veri kümesini döndürme adımları her sistemde aynıdır; bağlantı ve select komutlarının biçimi ise sisteme göre değişir. Hangi tasarım kalıbını seçerdiniz? Neden?

Cevap: Hocanın cevabı: Template Method. Strategy seçilirse farklı veritabanlarıyla çalışma karşılanır ama bu problemin yalnız yarısına hitap eder; çünkü her somut strateji işin tamamını yapar ve ortak adımlar her stratejide tekrar yazılır. Problemde işin ana hatları sabit, bazı ayrıntıları değişkendir. Bu yüzden ortak adımlar soyut QueryTemplate sınıfında final `doQuery` metodunda kodlanır, değişen `formatConnect` ve `formatSelect` soyut bırakılır ve OracleQT, SQLServerQT, GraphDBQT gibi alt sınıflarda doldurulur (ekrandan, 00:17:10).

Soru 2 (derste sözlü, 00:10:23): Template Method'un sıradan kalıtım ve override etmeden farkı nedir?

Cevap: Hocanın cevabı: kalıp tek bir sınıftan kalıtımı değil, birbiriyle ilişkili iki ayrı sınıftaki iki ayrı kalıtım hiyerarşisini (örnekte Document ve Application) birlikte ele alır. Ayrıca template metot override edilememelidir; bu yüzden soyut üst sınıftaki somut template metot final tanımlanır. Aksi halde bir alt sınıf onu override ederek framework'ün çalışma biçimini bozabilir.

Soru 3 (ekrandan, 00:36:50; derste sözlü, 00:37:05): Visitor kalıbı kurulmuş bir yapıya yeni bir ConcreteElement türü (ConcreteElementC) eklenirse neler yapmak gerekir? Bu, kalıbın hangi durumlarda kullanılması gerektiği konusunda ne söyler?

Cevap: Hocanın cevabı: Visitor arayüzüne `VisitConcreteElementC(ConcreteElementC)` metodu eklenir ve bütün somut ziyaretçiler, yeni elemanla işleri olmasa bile bu metodu (gerekirse boş gövdeyle) kodlamak zorunda kalır; yeni eleman da Accept içinde bu metodu çağırır. Bu yüzden yeni eleman türü eklemek zordur. Visitor, parça-bütün yapısının parçaları oturmuşsa ve değişen yalnız o parçalar üzerinde yapılacak işlerse uygulanmalıdır.

Soru 4 (derste sözlü, 01:44:31–01:46:02): Bir proje yönetim yazılımı hazırlanmaktadır; yazılımın şu şu bileşenleri vardır ve bu bileşenler üzerinde raporlamalar yapılacaktır. Raporlamaların ayrıntıları zamanla değişebilecektir ve bu yeni değişiklikler geriye doğru uyumluluk bozulmadan eklenmek istenmektedir. (a) Hangi tasarım kalıbını kullanırsınız? Literatürdeki adını yazın. (b) Bu kalıbı neden seçtiğinizi açıklayın. (c) Tasarımınızı ayrıntılı bir UML sınıf şeması ile gösterin. (d) Şemada kalıbın genel sınıf yapısı ve problem alanına özgü eklemeler görülebilmelidir.

> **Çözüm (derste verilmedi):** Hoca soruyu sınav sorusunun biçimini anlatmak için örnek olarak verdi; cevabı ayrıca söylemedi. Dersteki anlatıma göre uygun kalıp Visitor'dır: parça-bütün yapısı (Project, Task, Deliverable) oturmuştur ve değişen yalnız bu yapı üzerinde yapılacak işlerdir (raporlar); geriye doğru uyumluluğun uzun süre korunması da Visitor'ın mantıklı olduğu durumdur. Şema, dersteki Çözüm 2'deki gibi kurulabilir: soyut ProjectItem'da `accept(ProjectVisitor) {abstract}`; Project, Task ve Deliverable'da somut `accept`; `<<interface>> ProjectVisitor` içinde `visitProject(Project)`, `visitTask(Task)`, `visitDeliverable(Deliverable)`; her rapor için ProjectCostVisitor, ProjectTimeVisitor gibi ayrı bir somut ziyaretçi. Yeni bir rapor gerektiğinde yalnız yeni bir somut ziyaretçi eklenir, eleman sınıfları değişmez.

Soru 5 (derste sözlü, 01:41:06): Beş GUI bileşeni (colleague) birbiriyle doğrudan konuşursa kaç bağlantı olabilir, araya bir mediator konursa kaç bağlantı kalır? Bu, bağlaşımı azaltır mı?

Cevap: Hoca doğrudan bağlantı için "5 kere 5, 25" dedi, saymanın biçiminden emin olmadığını ekledi; mediator ile 5 ilişki kalacağını söyledi. Bir nesnenin kendisiyle bağlantısı sayılmazsa doğrudan bağlantı sayısı $5 \cdot 4/2 = 10$ (yönlü sayılırsa 20) olur (bkz. Bölüm 10.5'teki ders sonrası not). Hocanın cevabının ikinci kısmı: bağlaşım ilişkideki sınıf sayısıyla ölçüldüğünden sınıf düzeyinde bağlaşım değişmez; azalan yalnız ilişkideki nesne sayısıdır. Karmaşıklık ConcreteMediator'a taşınır ve o sınıf Large Class'a dönüşebilir.

Dersin Toparlaması

Bu derste davranışsal kalıpların son üçü işlendi ve böylece dönemdeki tasarım kalıpları tamamlandı. Template Method, bir algoritmanın iskeletini üst sınıftaki final bir metotta sabitleyip değişen adımları soyut metotlarla alt sınıflara bırakır; GoF'un belge açan çerçeve programı ve farklı veritabanlarından kayıt çeken QueryTemplate örneği bunu gösterdi. Hoca, işin tamamını değiştirilebilir kılan Strategy ile ana adımları sabitleyen Template Method arasındaki farkın doğru kalıbı seçmede belirleyici olduğunu vurguladı.

Visitor, oturmuş bir nesne yapısının elemanları üzerinde yeni işlemleri elemanların sınıflarını değiştirmeden eklemeyi sağlar; bedeli, düğümlerle ziyaretçi metotları arasındaki bağlaşım ve yeni eleman türü eklemenin zorluğudur. Proje yönetim yazılımı örneğinde instanceof'larla dolu ProjectCalculator yerine ProjectCostVisitor ve ProjectTimeVisitor kullanıldı; sonuç aynı kaldı, tür denetimleri kalktı ve sorumluluklar ayrıldı. Mediator ise bir nesne grubunun etkileşimlerini tek bir aracıda toplar; bağlantı sayısını azaltır ama sınıf düzeyinde bağlaşımı değiştirmez ve karmaşıklığı ConcreteMediator'a taşır. Hoca son olarak kalıp çözümlerinin sınıf şemalarında standart UML gösterimine (kalıtım oku, `<<abstract>>`, `{abstract}`, `{final}`) uyulmasını istedi ve sonraki derslerde kod kusurları ve refactoring konusuna geçileceğini söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.